

LAPORAN

PROYEK AKHIR

4143104 – MACHINE LEARNING

4143204 – BIG DATA



Disusun Oleh Kelompok 06:

NIM	NAMA
11423009	Odelia Josephine Simanjuntak
11423032	Ray Anugerah Tua Simarmata
11423040	Daniel Sinambela
11423042	Frangky Hamonangan Sirait
11423059	Venesa Herawaty Hutajulu

SARJANA TERAPAN TEKNOLOGI REKAYASA

PERANGKAT LUNAK (TRPL)

INSTITUT TEKNOLOGI DEL

2026

BAB I PENDAHULUAN

1.1 Deskripsi Proyek

Proyek ini bertujuan untuk membangun sistem web agent terpadu berbasis Laravel 12 yang mengotomatisasi seluruh siklus administrasi dan penjaminan mutu pada Gugus Kendali Mutu (GKM) dan Gugus Jaminan Mutu (GJM) di Fakultas Vokasi Institut Teknologi Del. Sistem ini mengintegrasikan dua pilar teknologi cerdas: Pipeline Big Data Terdistribusi untuk pengolahan data kuesioner skala besar, dan Pipeline NLP-RAG (Retrieval-Augmented Generation) untuk otomatisasi dokumen naratif mutu akademik.

Secara end-to-end, arsitektur sistem mencakup lima tahapan siklus data, yaitu:

- Ingestion: Pengambilan data transaksional terstruktur dari database MySQL Campus Information System (CIS) melalui konektor Spark JDBC, serta ekstraksi dokumen digital/fisik menggunakan OCR berbasis deep learning.
- Processing: Distributed batch processing berbasis Apache Spark (PySpark) untuk pembersihan data (regex cleaning), pemetaan skor, serta advanced semantic chunking teks.
- Storage: Penyimpanan hibrida yang mengintegrasikan database relasional (MySQL) untuk data master transaksional dan non-relasional (MongoDB/Vector DB) untuk menyimpan hasil analisis agregat dan vector embeddings.
- Analisis: Pemodelan analitik cerdas yang mencakup statistik deskriptif kuesioner, pattern mining, serta pemrosesan NLP-RAG untuk pembuatan laporan otomatis.
- Visualisasi: Dashboard monitoring interaktif real-time berbasis Laravel dan Chart.js yang menyajikan tren kualitas pembelajaran serta fitur unduh dokumen otomatis.

1.2 Latar Belakang

Institusi pendidikan tinggi diwajibkan untuk menyusun laporan akademik secara berkala sebagai pemenuhan Sistem Penjaminan Mutu Internal (SPMI). GKM dan GJM memegang peranan krusial dalam mengevaluasi ribuan data respons kuesioner serta menyusun berbagai dokumen mutu, seperti laporan bulanan, laporan kuesioner mahasiswa, laporan triwulan, laporan semester, hingga laporan VMTS.

Namun dalam praktiknya, proses ini menghadapi sejumlah kendala kritical:

- Inefisiensi Alur Kerja Manual: Proses analisis data evaluasi kuantitatif dan penyusunan narasi laporan kualitatif masih mengandalkan Microsoft Excel dan email, memakan waktu rata-rata 3,5 jam per laporan, rentan terhadap kesalahan, dan tidak memiliki penyimpanan terpusat.
- Keterbatasan Skalabilitas Database Relasional: Query analitik kompleks pada database SQL konvensional mengalami degradasi performa saat memproses data kuesioner historis yang volumenya melebihi 100.000 baris.
- Karakter Data Semi-Terstruktur yang Tidak Terkelola: Umpan balik mahasiswa berupa saran teks bebas, metadata dokumen RPS, dan berkas VMTS bersifat semi-terstruktur, sehingga sulit dianalisis secara optimal menggunakan struktur tabel SQL murni.

- Ketidadaan Validasi dan Ringkasan Otomatis: Belum adanya mekanisme monitoring real-time untuk memetakan sentimen kepuasan mahasiswa secara visual, serta tidak adanya alat bantu cerdas untuk mengekstrak dan mensintesis informasi dari dokumen secara cepat.

Untuk mengatasi tantangan tersebut, proyek ini mengintegrasikan Apache Spark untuk pemrosesan batch terdistribusi, MySQL untuk data terstruktur, MongoDB untuk dokumen analitik, serta teknik NLP dan RAG menggunakan model bahasa besar (LLM) Groq LLaMA. Sistem ini mampu menghasilkan narasi laporan akademik yang akurat dan kontekstual dalam waktu kurang dari 5 menit, memangkas waktu dari rata-rata 3,5 jam menjadi di bawah 5 menit.

1.3 Tujuan Penelitian

- Membangun sistem web agent terpadu berbasis Laravel 12 untuk otomatisasi pengelolaan administrasi pada GKM dan GJM Fakultas Vokasi Institut Teknologi Del.
- Mengembangkan pipeline Big Data end-to-end menggunakan distributed batch processing Apache Spark (PySpark) untuk mengolah, membersihkan, dan menganalisis data evaluasi kuesioner mahasiswa berskala besar (>100.000 records).
- Mengimplementasikan pipeline NLP-RAG (Retrieval-Augmented Generation) terstruktur, mulai dari tahapan OCR berbasis deep learning, ekstraksi file, advanced semantic chunking, pencarian semantik (vector search), hingga generasi narasi laporan.
- Mengintegrasikan arsitektur database hibrida dengan memadukan MySQL untuk data transaksional terstruktur dan MongoDB untuk dokumen penyimpanan hasil analisis agregat terdistribusi.
- Mengevaluasi kualitas output sistem menggunakan metrik NLP yang relevan, termasuk RAGAS evaluation dan ambiguity evaluation.

BAB II IMPLEMENTASI MACHINE LEARNING

2.1. Problem Definition

2.1.1. Latar belakang masalah berdasarkan domain yang dipilih

Penyusunan dokumen penjaminan mutu akademik di lingkungan Fakultas Vokasi Institut Teknologi Del selama ini masih dilakukan secara manual. Proses ini melibatkan pengelolaan data evaluasi dari ratusan mata kuliah yang tersebar lintas program studi, sehingga tim GKM dan GJM harus membaca, merangkum, dan menyusun narasi dari ribuan baris data kuesioner dan dokumen laporan setiap periodenya. Selain memakan waktu yang signifikan, proses manual ini juga rentan terhadap inkonsistensi gaya penulisan dan kesalahan interpretasi data.

Lima jenis laporan yang wajib disusun secara berkala adalah: Laporan Bulanan (GKM), Laporan Kuesioner (GKM), Laporan Triwulan (GJM), Laporan Semester (GJM), dan Laporan VMTS (GJM). Masing-masing laporan memerlukan pengolahan dokumen dan data yang berbeda, sehingga beban kerja administratif menjadi sangat tinggi tanpa adanya sistem otomasi.

2.1.2. Tujuan pengembangan model Machine Learning

- a. Mengotomatisasi penyusunan dokumen akademik mutu (GKM/GJM) secara end-to-end menggunakan pipeline NLP-RAG berbasis LLM.
- b. Meningkatkan efisiensi dan konsistensi laporan dengan mengurangi waktu penyusunan dari rata-rata 3,5 jam menjadi di bawah 5 menit per laporan.
- c. Membangun sistem cerdas yang mampu memahami instruksi pengguna dalam bahasa alami (NLP) dan mengambil konteks dokumen relevan (RAG) untuk menghasilkan narasi laporan yang akurat dan kontekstual.

2.1.3. Alasan pemilihan topik penelitian dan algoritma

Topik ini dipilih karena relevan langsung dengan kebutuhan nyata institusi. Domain penjaminan mutu pendidikan memerlukan pengelolaan dokumen teks dalam jumlah besar secara periodik, menjadikannya kasus penggunaan ideal untuk teknik NLP modern.

Algoritma NLP dan RAG dipilih dengan alasan sebagai berikut:

- NLP digunakan untuk memahami instruksi pengguna dalam bahasa alami dan memproses teks dokumen akademik melalui tahapan text cleaning, tokenization, dan embedding generation.
- RAG dipilih karena kemampuannya menggabungkan pencarian informasi semantik dari knowledge base lokal dengan kemampuan generasi teks LLM, menghasilkan output yang faktual dan terkonteks tanpa halusinasi berlebihan.

- Pendekatan RAG lebih cocok dibandingkan fine-tuning LLM langsung karena tidak memerlukan data pelatihan berlabel dalam jumlah besar dan dapat dengan mudah diperbarui dengan dokumen-dokumen terbaru.

2.2. Workflow Agent-Based Report Assistant

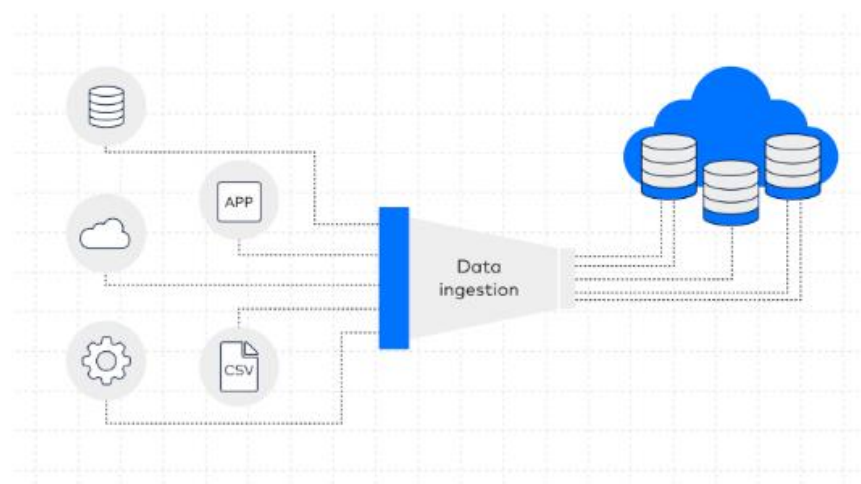
2.2.1. Data Collection

Pengumpulan dokumen merupakan tahap awal dalam workflow *Agent-Based Report Assistant* yang berfungsi sebagai proses penyediaan sumber informasi akademik yang akan digunakan sebagai basis pengetahuan (*knowledge base*) sistem. Dokumen yang terkumpul menjadi sumber data yang digunakan dalam arsitektur RAG untuk mendukung proses pencarian informasi dan penyusunan laporan.

Dokumen yang dikelola pada tahap ini meliputi data kuesioner kepuasan mahasiswa, data monitoring perkuliahan, Rencana Pembelajaran Semester (RPS), materi perkuliahan, data penugasan dosen, serta dokumen pendukung lainnya yang berkaitan dengan kegiatan akademik. Sumber dokumen dirancang dapat berasal dari unggah dokumen oleh pengguna maupun integrasi dengan CIS Del.

Tahap pengumpulan dokumen difokuskan pada penyediaan dan pengorganisasian dokumen sebelum memasuki proses pengolahan lebih lanjut. Dokumen yang telah terkumpul selanjutnya akan digunakan pada tahap Data Ingestion untuk mempersiapkan data agar dapat diproses dalam tahapan berikutnya pada workflow *Agent-Based Report Assistant*.

2.2.2. Data Ingestion



Gambar diatas menampilkan Data Ingestion, Data Ingestion merupakan tahap yang dirancang untuk mempersiapkan dokumen akademik sebelum digunakan sebagai sumber pengetahuan (*knowledge base*) pada *Agent-Based Report Assistant*. Dokumen yang dikelola pada tahap ini meliputi standar mutu, laporan audit mutu internal, borang evaluasi,

pedoman akademik, serta dokumen pendukung lainnya dalam format PDF, DOCX, XLSX, JPG, PNG maupun dokumen hasil pemindaian (*scanned document*).

Pada rancangan sistem, dokumen hasil pemindaian dapat dikonversi menjadi teks digital melalui mekanisme *Optical Character Recognition* (OCR) sehingga informasi yang terkandung di dalamnya dapat diproses lebih lanjut. Selanjutnya, dokumen akan melalui proses ekstraksi, pembersihan, dan transformasi data untuk menghasilkan format yang lebih konsisten dan sesuai dengan kebutuhan pengolahan data. Hasil dari tahap Data Ingestion berupa data teks yang telah terstruktur dan siap digunakan pada proses chunking, pembentukan embedding, serta penyimpanan pada *vector store* sebagai bagian dari arsitektur RAG.

2.2.3. Chunking



Pada Gambar diatas menampilkan Proses Chunking, Chunking merupakan tahap yang dirancang untuk membagi dokumen yang telah melalui proses Data Ingestion menjadi beberapa bagian teks yang lebih kecil (*chunks*). Setiap *chunk* merepresentasikan bagian informasi tertentu dari dokumen asli sehingga informasi dapat dikelola secara lebih efektif pada proses selanjutnya. Pembagian dokumen ke dalam beberapa *chunk* bertujuan untuk mengurangi jumlah teks yang diproses dalam satu waktu, mempermudah proses pencarian informasi, serta meningkatkan akurasi *retrieval* pada arsitektur RAG. Dengan pendekatan ini, sistem dapat menemukan informasi yang lebih relevan berdasarkan konteks pertanyaan yang diberikan oleh pengguna. Hasil dari tahap chunking berupa kumpulan *chunk* yang akan digunakan pada proses pembentukan embedding dan penyimpanan ke dalam *vector store*.

2.2.4. Embedding Generation



Pada Gambar diatas menampilkan Embedding Generation merupakan tahap yang dirancang untuk mengubah setiap *chunk* menjadi representasi numerik dalam bentuk vektor (*embedding*). Proses ini bertujuan agar informasi yang terdapat pada dokumen dapat direpresentasikan dalam format yang dapat dipahami dan diproses oleh sistem. Melalui proses *embedding*, setiap *chunk* tidak hanya direpresentasikan berdasarkan kata-kata yang terkandung di dalamnya, tetapi juga berdasarkan makna, hubungan antar kata, dan konteks informasi yang dimiliki. Dengan demikian, *chunk* yang memiliki makna serupa akan memiliki representasi vektor yang saling berdekatan meskipun menggunakan susunan kata yang berbeda. Hasil dari tahap ini berupa kumpulan vektor (*embedding*) yang merepresentasikan isi dari setiap *chunk*. Vektor tersebut selanjutnya akan digunakan pada proses penyimpanan ke *vector store* dan pencarian informasi berdasarkan kemiripan makna (*semantic similarity*).

2.2.5. Penyimpanan ke Vector Store

Penyimpanan ke *Vector Store* merupakan tahap yang dirancang untuk menyimpan hasil *embedding* yang telah dihasilkan dari setiap *chunk*. Selain vektor, informasi pendukung seperti metadata dan referensi dokumen juga disimpan untuk menjaga keterkaitan antara data dan sumber aslinya. *Vector Store* berfungsi sebagai basis pengetahuan yang mendukung proses pencarian informasi berdasarkan kemiripan makna (*semantic similarity*). Pada rancangan sistem ini, penyimpanan data direncanakan menggunakan MongoDB Atlas sebagai penyimpanan data dokumen dan *embedding*, serta MySQL untuk mengelola data operasional aplikasi. Data yang tersimpan pada tahap ini akan digunakan pada proses *retrieval* untuk menemukan informasi yang paling relevan terhadap pertanyaan atau permintaan pengguna.

2.2.6. Pengajuan Pertanyaan oleh Pengguna

Pengajuan pertanyaan oleh pengguna merupakan tahap yang memungkinkan pengguna berinteraksi dengan *Agent-Based Report Assistant* melalui antarmuka sistem. Pada tahap ini, pengguna dapat memasukkan pertanyaan atau permintaan informasi yang berkaitan dengan dokumen dan data akademik yang tersedia pada basis pengetahuan sistem. Pertanyaan yang diajukan dapat berupa permintaan informasi terkait monitoring RPS, monitoring pelaksanaan perkuliahan, pengingat (*reminder*) terhadap aktivitas akademik

tertentu, maupun pembuatan laporan berdasarkan data dan dokumen yang tersedia. Pertanyaan tersebut selanjutnya akan diproses pada tahap berikutnya untuk menemukan informasi yang paling relevan sebelum menghasilkan respons kepada pengguna.

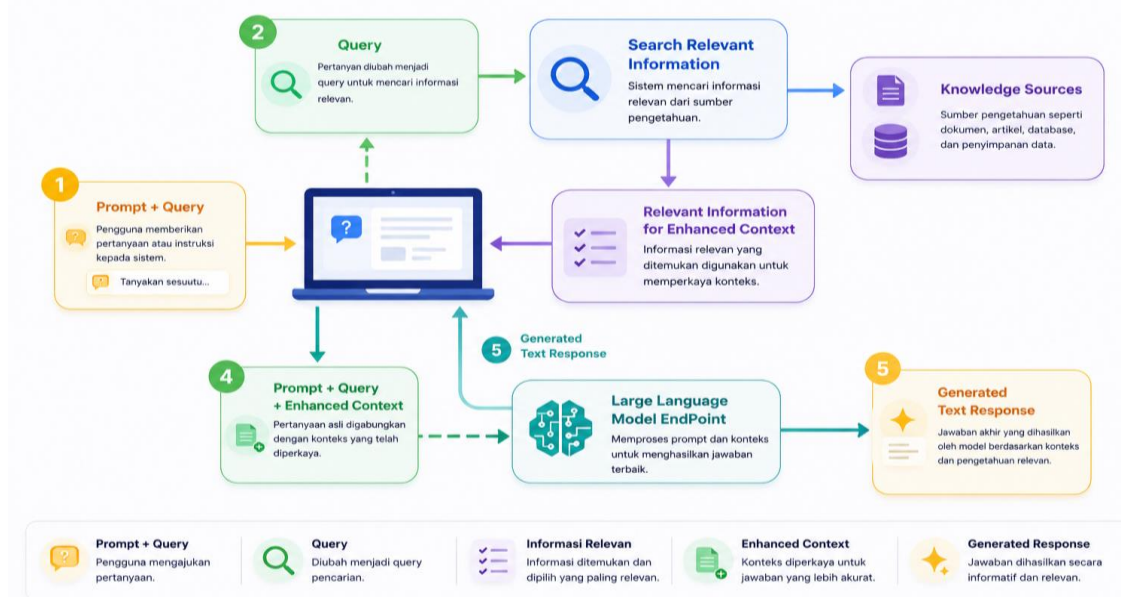
2.2.7. Query Embedding



Pada Gambar diatas menampilkan Query Embedding, Pada tahap Query Embedding, pertanyaan atau permintaan yang diberikan oleh pengguna diproses oleh Query Embedding Module untuk diubah menjadi representasi numerik dalam bentuk vector embedding. Proses ini menggunakan model embedding yang sama dengan yang digunakan pada dokumen di dalam *Vector Store*, sehingga pertanyaan pengguna dan dokumen yang tersimpan berada dalam ruang vektor yang sama dan dapat dibandingkan secara matematis.

Vector embedding yang dihasilkan kemudian digunakan sebagai dasar untuk melakukan proses pencarian informasi yang relevan pada *Vector Store*. Sistem akan menghitung tingkat kemiripan (*similarity*) antara embedding pertanyaan dengan embedding dokumen yang telah tersimpan, kemudian mengambil *chunks* yang memiliki tingkat relevansi tertinggi. Hasil pencarian tersebut akan diteruskan ke tahap *retrieval* sebagai konteks yang digunakan oleh model bahasa untuk menghasilkan jawaban yang sesuai dengan kebutuhan pengguna.

2.2.8. Retrieval Process



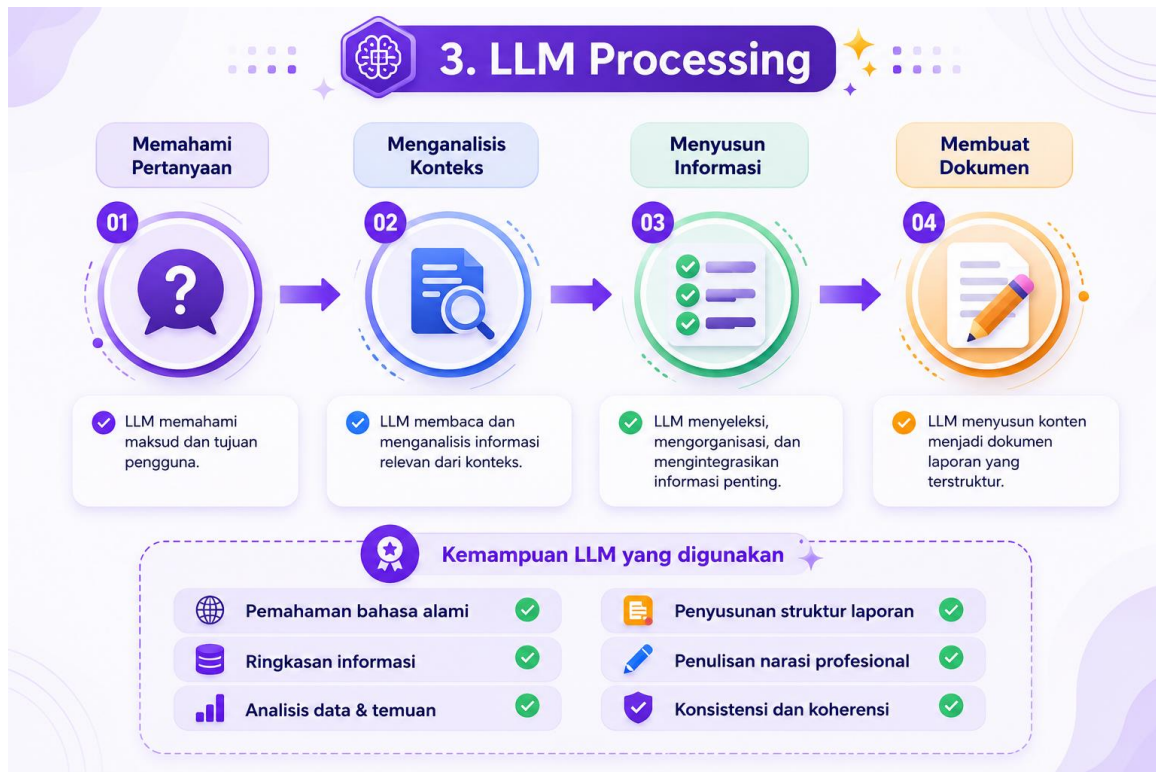
Pada Gambar diatas menampilkan Retrieval Process, Retrieval Process merupakan tahap yang dirancang untuk menemukan informasi yang paling relevan terhadap pertanyaan atau permintaan yang diajukan pengguna. Pada tahap ini, *query embedding* yang telah dihasilkan sebelumnya digunakan untuk melakukan pencarian pada *Vector Store*. Proses pencarian dilakukan dengan membandingkan kemiripan antara *query embedding* dan *embedding* dokumen yang tersimpan pada *Vector Store*. Hasil dari proses ini berupa kumpulan *chunk* yang memiliki tingkat relevansi tertinggi dan akan digunakan sebagai konteks pada tahap pembentukan respons oleh Large Language Model (LLM).

2.2.9. Relevant Context Construction

Pada tahap Relevant Context Construction, sistem mengumpulkan dan menyusun *chunks* dokumen yang diperoleh dari proses retrieval berdasarkan tingkat relevansinya terhadap pertanyaan pengguna. *Chunks* yang terpilih dapat berasal dari berbagai dokumen yang tersimpan dalam basis pengetahuan, seperti dokumen desain, dokumen produk, standar mutu, pedoman, maupun dokumen pendukung lainnya yang berkaitan dengan kebutuhan pengguna. Informasi yang telah ditemukan kemudian diorganisasikan dan digabungkan sehingga membentuk konteks yang utuh dan mudah dipahami.

Konteks yang telah terbentuk berfungsi sebagai sumber informasi utama yang akan diberikan kepada Large Language Model (LLM) pada tahap generasi respons. Dengan menggunakan konteks yang relevan, sistem dapat menghasilkan jawaban yang lebih akurat, memberikan ringkasan informasi yang sesuai, serta merekomendasikan dokumen desain atau dokumen produk yang paling berkaitan dengan pertanyaan pengguna. Proses ini membantu memastikan bahwa respons yang dihasilkan tidak hanya berdasarkan pengetahuan umum model, tetapi juga berdasarkan dokumen yang tersedia pada sistem GKM dan GJM.

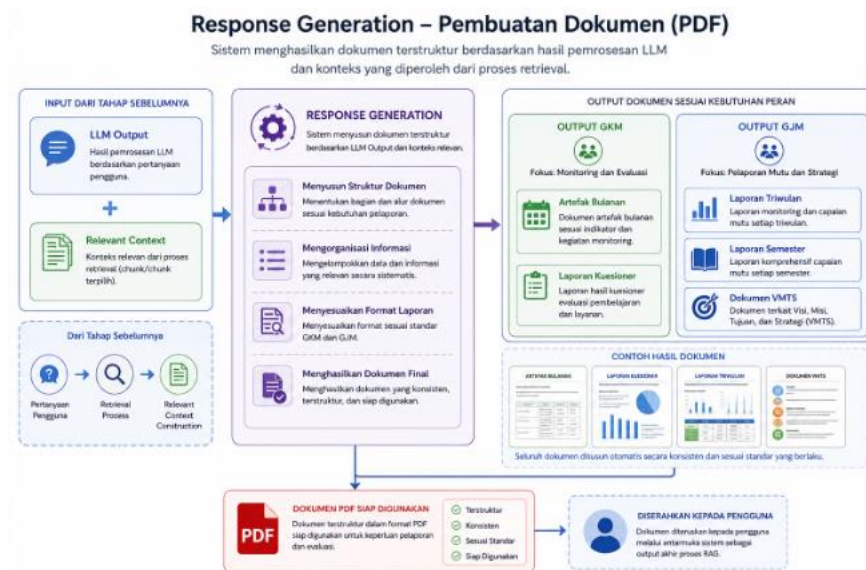
2.2.10. LLM Processing



Pada Gambar 3.14 menampilkan LLM Processing, pada tahap LLM Processing, sistem mengolah pertanyaan pengguna bersama dengan konteks yang telah disusun pada tahap *Relevant Context Construction*. Pada tahap ini, LLM digunakan untuk memahami maksud pertanyaan, menganalisis informasi dari konteks yang tersedia, serta mengolahnya menjadi keluaran dalam bentuk dokumen yang terstruktur sesuai kebutuhan sistem GKM dan GJM.

Berbeda dengan sistem tanya-jawab biasa, keluaran pada tahap ini tidak hanya berupa jawaban langsung, tetapi berupa dokumen hasil pemrosesan yang dapat mencakup ringkasan terstruktur, laporan, atau kompilasi informasi dari berbagai sumber dokumen yang relevan. Dengan demikian, LLM berperan dalam menyusun kembali informasi menjadi bentuk dokumen yang lebih formal, konsisten, dan sesuai dengan kebutuhan pelaporan atau analisis pada sistem berbasis RAG.

2.2.11. Response Generation



Pada Gambar diatas menampilkan Response Generation, sistem menghasilkan keluaran akhir berdasarkan hasil pemrosesan oleh Large Language Model (LLM). Pada konteks implementasi sistem GKM dan GJM, respons yang dihasilkan tidak hanya berupa jawaban singkat, tetapi berupa dokumen terstruktur yang sesuai dengan kebutuhan masing-masing peran. Sistem memanfaatkan pertanyaan pengguna dan konteks hasil retrieval untuk memastikan bahwa dokumen yang dihasilkan tetap selaras dengan sumber pengetahuan yang tersedia dalam sistem.

Bentuk dokumen yang dihasilkan disesuaikan dengan kebutuhan operasional, yaitu pada GKM berupa artefak bulanan dan laporan kuesioner, sedangkan pada GJM berupa laporan triwulan, laporan semester, serta dokumen terkait VMTS. Seluruh dokumen tersebut disusun secara otomatis dalam format yang terstruktur, konsisten, dan siap digunakan untuk keperluan pelaporan maupun evaluasi. Hasil dari tahap ini kemudian diteruskan kepada pengguna melalui antarmuka sistem sebagai output akhir dari proses RAG.

2.3. Implementasi

Implementasi AI *Agent* dan NLP-RAG (Retrieval-Augmented Generation) pada sistem ini digunakan untuk mendukung proses analisis data dan pembuatan laporan secara otomatis pada berbagai fitur pelaporan, seperti analisis kuesioner, laporan bulanan, laporan kuesioner, laporan triwulan, laporan semester, laporan VMTS, serta pembuatan presentasi. Teknologi ini memanfaatkan mekanisme retrieval untuk memperoleh informasi dan konteks yang relevan dari data yang tersedia sebelum diproses oleh Large Language Model (LLM), sehingga laporan, analisis, dan rekomendasi yang dihasilkan lebih sesuai dengan kebutuhan pengguna dan kondisi data yang dimiliki sistem.

2.2.12. Data Cleaning

```

/**
 * Check if command exists
 */
private function commandExists(string $command): bool
{
    $os = strtoupper(substr(PHP_OS, 0, 3));

    if ($os === 'WIN') {

```

```

        exec("where {$command}", $output, $returnCode);
    } else {
        exec("which {$command}", $output, $returnCode);
    }

    return $returnCode === 0;
}

/**
 * Clean extracted text
 */
public function cleanText(string $text): string
{
    // Remove excessive whitespace
    $text = preg_replace('/[ \t]+/', ' ', $text);

    // Normalize line breaks
    $text = preg_replace('/\r\n|\r/', "\n", $text);

    // Remove more than 2 consecutive line breaks
    $text = preg_replace('/\n{3,}/', "\n\n", $text);

    // IMPORTANT: Escape curly braces to prevent template string
errors
    // This prevents "Unclosed '{' on line X" errors when PDF
content contains { or }
    $text = str_replace(['{', '}'], ['{{', '}}'], $text);

    // Trim
    $text = trim($text);

    return $text;
}

```

Pada Gambar diatas menjelaskan Tahap cleaning dilakukan oleh komponen TextExtractionService.php dan ChunkingService.php. Proses ini mencakup penghapusan karakter noise hasil OCR, normalisasi whitespace dan line breaks, serta penghapusan baris kosong berulang. Berikut adalah potongan kode implementasi cleaning.

2.2.13. Tokenization

```

/**
 * Split text into sentences
 */
private function splitIntoSentences(string $text): array
{
    // Split by common sentence endings

```

```

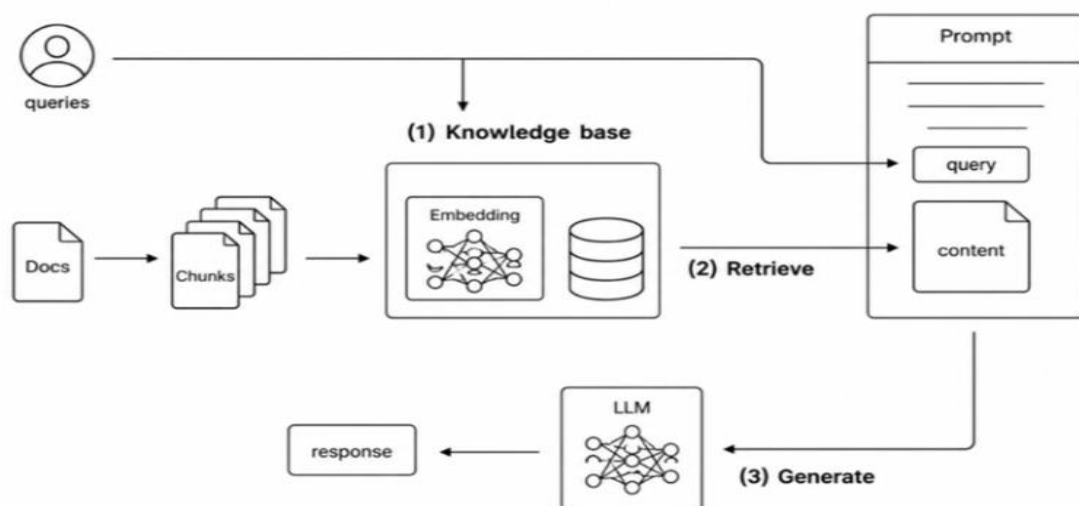
        $sentences = preg_split('/(?<=[.!?])\s+/', $text, -1,
PREG_SPLIT_NO_EMPTY);

        return array_filter($sentences, function($sentence) {
            return strlen(trim($sentence)) > 10; // Minimum sentence
length
        });
    }

```

Pada Gambar diatas menjelaskan Tokenization dilakukan dengan membagi teks menjadi unit-unit kalimat menggunakan fungsi splitIntoSentences() dalam ChunkingService.php. Teks dipecah berdasarkan tanda akhir kalimat menggunakan regex pattern, dan kalimat dengan panjang di bawah 10 karakter dibuang untuk menghilangkan fragmen yang tidak bermakna

2.2.14. Konsep RAG



Pada Gambar diatas mejelaskan Pipeline RAG yang diterapkan pada sistem ini. Retrieval-Augmented Generation (RAG) adalah teknik yang menggabungkan kemampuan retrieval (pengambilan informasi) dengan generasi teks dari LLM.

1. Chunking

```

/**
 * Split text into chunks with overlap
 */
public function chunkText(string $text): array
{
    // Clean text first
    $text = $this->cleanText($text);

    // Split by sentences first
    $sentences = $this->splitIntoSentences($text);

    $chunks = [];
    $currentChunk = '';
    $chunkIndex = 0;

```

```

        foreach ($sentences as $sentence) {
            $testChunk = $currentChunk . ' ' . $sentence;

            if (strlen($testChunk) > $this->chunkSize &&
!empty($currentChunk)) {
                // Save current chunk
                $chunks[] = [
                    'text' => trim($currentChunk),
                    'index' => $chunkIndex++,
                    'length' => strlen($currentChunk),
                ];

                // Start new chunk with overlap
                $words = explode(' ', $currentChunk);
                $overlapWords = array_slice($words, -10); // Last 10
words as overlap
                $currentChunk = implode(' ', $overlapWords) . ' ' .
$sentence;
            } else {
                $currentChunk = $testChunk;
            }
        }

        // Add last chunk
        if (!empty(trim($currentChunk))) {
            $chunks[] = [
                'text' => trim($currentChunk),
                'index' => $chunkIndex,
                'length' => strlen($currentChunk),
            ];
        }
    }

```

Pada Gambar diatas menampilkan implementasi Chunking yang dimana dokumen yang diunggah dipecah menjadi potongan-potongan kecil (chunks) oleh ChunkingService.php. Setiap chunk memiliki ukuran sekitar 500 token dengan overlap 50 token antar chunk. Metadata chunk mencakup: source_document, chunk_index, page_number, section_title, dan document_type untuk mendukung filtering saat retrieval.

2. Embedding

```

public function generateEmbedding(string $text): ?array
{
    // Check cache first
    $cached = EmbeddingsCache::getCached($text, $this->model);
    if ($cached) {

```

```

        Log::info("Embedding cache hit", ['text_length' =>
strlen($text)]);
        return $cached;
    }

    // Generate new embedding
    try {
        Log::info("Generating new embedding", [
            'provider' => $this->provider,
            'model' => $this->model,
            'text_length' => strlen($text)
        ]);

        $embedding = $this->callEmbeddingAPI($text);

        if ($embedding) {
            // Cache for future use
            EmbeddingsCache::store($text, $embedding, $this->model);
            return $embedding;
        }

        return null;
    } catch (\Exception $e) {
        Log::error("Embedding generation failed", [
            'error' => $e->getMessage()
        ]);
        return null;
    }
}

/**
 * Generate embeddings for multiple texts (batch)
 */
public function generateEmbeddings(array $texts): array
{
    $embeddings = [];

    foreach ($texts as $index => $text) {
        $embedding = $this->generateEmbedding($text);
        if ($embedding) {
            $embeddings[$index] = $embedding;
        }
    }

    return $embeddings;
}

```

Pada Gambar diatas menampilkan implementasi Embedding yang dimana setiap chunk dikonversi menjadi vector embedding oleh EmbeddingService.php. Embedding ini merepresentasikan makna semantik dari teks dalam ruang vektor berdimensi tinggi. Teks yang bermakna mirip akan memiliki representasi vektor yang berdekatan secara geometris, memungkinkan pencarian berbasis kemiripan semantik (bukan hanya kata kunci).

3. Vector Database

```
public function search(string $query, int $topK = 10, array
$filters = []): array
{
    try {
        Log::info("=== Vector Search ===", [
            'query' => substr($query, 0, 100),
            'top_k' => $topK,
            'filters' => $filters
        ]);

        // Generate query embedding
        $queryEmbedding = $this->embeddingService-
>generateEmbedding($query);

        if (!$queryEmbedding) {
            Log::error("Failed to generate query embedding");
            return [];
        }

        // Get all chunks (with filters)
        $chunksQuery = DocumentChunk::with('kuesioneUpload');

        // Apply filters
        if (isset($filters['source_type'])) {
            $chunksQuery->where('source_type',
$filters['source_type']);
        }

        if (isset($filters['source_id'])) {
            $chunksQuery->where('source_id',
$filters['source_id']);
        }

        if (isset($filters['periode'])) {
            $chunksQuery->whereHas('kuesioneUpload',
function($q) use ($filters) {
                $q->whereRaw("DATE_FORMAT(created_at, '%Y-%m') =
?", [$filters['periode']]);
            });
        }
    }
}
```

```

    }

    if (isset($filters['prodi_id'])) {
        $chunksQuery->whereHas('kuesioneUpload.user',
function($q) use ($filters) {
            $q->where('prodi_id', $filters['prodi_id']);
        });
    }

    $chunks = $chunksQuery->get();

    Log::info("Chunks retrieved for search", [
        'total_chunks' => $chunks->count()
    ]);

    // Calculate similarities
    $results = [];
    foreach ($chunks as $chunk) {
        if ($chunk->embedding) {
            $similarity = $this->embeddingService-
>cosineSimilarity(
                $queryEmbedding,
                $chunk->embedding
            );

            $results[] = [
                'chunk' => $chunk,
                'similarity' => $similarity,
                'text' => $chunk->chunk_text,
                'metadata' => $chunk->metadata,
            ];
        }
    }

    // Sort by similarity (descending)
    usort($results, function($a, $b) {
        return $b['similarity'] <=> $a['similarity'];
    });

    // Return top K
    $topResults = array_slice($results, 0, $topK);

    Log::info("Search completed", [
        'results_found' => count($results),
        'top_k_returned' => count($topResults),
        'top_similarity' => $topResults[0]['similarity'] ??
0
    ]);

```

```

        return $topResults;

    } catch (\Exception $e) {
        Log::error("Vector search failed", [
            'error' => $e->getMessage()
        ]);
        return [];
    }
}

```

Pada Gambar diatas menampilkan implementasi Vector embedding disimpan ke dalam Vector Database (VectorDatabaseService.php) yang berfungsi sebagai knowledge base sistem. Database ini mengindeks embedding beserta metadata dokumen aslinya, sehingga pencarian dapat dilakukan secara efisien menggunakan cosine similarity.

4. Retrieve

```

public function retrieveContext(string $query, array $filters = []):
array
{
    Log::info("=== RAG Retrieval ===", [
        'query' => substr($query, 0, 100),
        'filters' => $filters
    ]);

    // Search vector database
    $results = $this->vectorDbService->search($query, $this-
>topK, $filters);

    // Filter by similarity threshold
    $relevantResults = array_filter($results, function($result)
{
        return $result['similarity'] >= $this-
>similarityThreshold;
    });

    Log::info("Context retrieved", [
        'total_results' => count($results),
        'relevant_results' => count($relevantResults),
        'avg_similarity' => count($relevantResults) > 0
            ? array_sum(array_column($relevantResults,
'similarity')) / count($relevantResults)
            : 0
    ]);

    return [
        'results' => $relevantResults,

```

```

        'context_text' => $this->buildContextText($relevantResults),
        'metadata' => $this->extractMetadata($relevantResults),
    ];
}

/**
 * Build context text from results
 */
private function buildContextText(array $results): string
{
    if (empty($results)) {
        return '';
    }

    $contextParts = [];

    foreach ($results as $index => $result) {
        // Check if 'text' key exists
        if (!isset($result['text'])) {
            Log::warning("Result missing 'text' key", [
                'index' => $index,
                'result_keys' => array_keys($result)
            ]);
            continue;
        }

        $similarity = round($result['similarity'] * 100, 1);
        $contextParts[] = "--- Context {$index} (Relevance: {$similarity}%) ---\n{$result['text']}\n";
    }

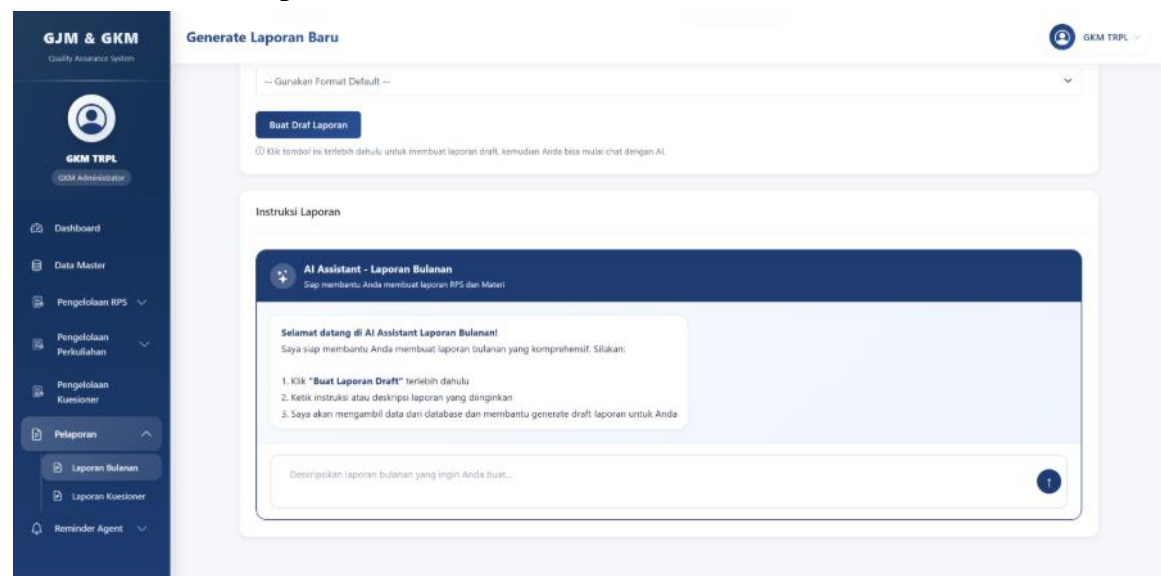
    return implode("\n", $contextParts);
}

```

Pada Gambar 4.50 menampilkan implemtasi Retrieve saat pengguna mengajukan query, sistem RAG RetrievalService.php melakukan hal berikut:

- Mengonversi query menjadi embedding menggunakan model yang sama.
- Melakukan vector search pada database untuk menemukan top-K chunks yang paling relevan (threshold similarity $\geq 0,85$ setelah optimasi).
- Memfilter hasil berdasarkan similarity threshold untuk memastikan hanya konteks yang benar-benar relevan yang digunakan.
- Membangun context text dari chunk-chunk terpilih untuk disertakan dalam prompt ke LLM.

2.3.1. Generate Laporan Otomatis



Gambar diatas menampilkan Generate Laporan Otomatis diimplementasikan untuk membantu proses penyusunan berbagai jenis laporan akademik secara otomatis dengan memanfaatkan *AI Agent* berbasis NLP-RAG. Pada fitur ini, sistem mengambil data dan dokumen yang relevan dari berbagai sumber, kemudian melakukan proses retrieval untuk memperoleh konteks yang sesuai sebelum diproses oleh Large Language Model (LLM). Hasil pemrosesan tersebut digunakan untuk menghasilkan bagian-bagian laporan seperti latar belakang, hasil analisis, pembahasan, kesimpulan, dan rekomendasi secara otomatis sesuai dengan template yang dipilih pengguna. Implementasi fitur ini diterapkan pada pembuatan laporan bulanan, laporan kuesioner, laporan triwulan, laporan semester, dan laporan VMTS, sehingga proses penyusunan laporan yang sebelumnya dilakukan secara manual dapat dilakukan secara lebih terstruktur dan konsisten.

2.4. Evaluasi RAGAS

Evaluasi sistem *Retrieval-Augmented Generation* (RAG) dilakukan untuk mengukur kualitas proses *retrieval* dan *generation* yang digunakan dalam pembuatan laporan otomatis. Pengujian ini bersifat ilmiah dan kuantitatif menggunakan *framework* RAGAS (*Retrieval-Augmented Generation Assessment*), yaitu framework evaluasi *open-source* yang menyediakan serangkaian metrik terstandar untuk mengukur performa sistem RAG secara menyeluruh.

Framework RAGAS dipilih karena mampu mengevaluasi dua aspek utama sistem RAG secara independen, yaitu kualitas proses *retrieval* yang mengukur seberapa tepat sistem dalam mengambil konteks yang relevan, serta kualitas proses *generation* yang mengukur seberapa akurat dan relevan jawaban yang dihasilkan model terhadap dokumen sumber. Pengujian dilakukan pada fitur *AI Assistant* yang digunakan dalam proses pembuatan laporan akademik, meliputi:

1. Laporan Bulanan (monitoring RPS dan materi perkuliahan)
2. Laporan Kuesioner (analisis kepuasan mahasiswa)
3. Laporan Triwulan

4. Laporan Semester
5. Laporan VMTS (Visi, Misi, Tujuan, dan Sasaran)

Metrik RAGAS (Retrieval-Augmented Generation Assessment) adalah sekumpulan metrik evaluasi khusus yang dirancang untuk mengukur dan menguji kinerja sistem kecerdasan buatan berbasis LLM (seperti chatbot atau asisten AI) yang menggunakan dokumen eksternal untuk menjawab pertanyaan.

1. *Faithfulness*

Faithfulness mengukur kesesuaian jawaban yang dihasilkan sistem dengan dokumen sumber. Setiap pernyataan dalam jawaban harus dapat ditelusuri kebenarannya ke dokumen konteks yang diberikan.

Rumus :

$$F = \frac{P_d}{P_t} \times 100 \%$$

Keterangan :

F = *Faithfulness* (%)

P_d = Jumlah pernyataan yang didukung dokumen sumber

P_t = Total seluruh pernyataan dalam jawaban

Berdasarkan pada pertanyaan buatlah laporan triwulan terbaru terdapat pada Gambar 5.5 hasil pengujian diperoleh rata-rata *Faithfulness* sebesar:

P_d = 14 pernyataan didukung dokumen

P_t = 19 total pernyataan

$$F = \frac{P_d}{P_t} \times 100 \%$$

$$F = \frac{14}{19} \times 100 \%$$

$$F = 73,68 \% \approx 74 \%$$

Pada Tabel 5.32 menjelaskan Kriteria Penilaian *Faithfulness*:

Tabel 5.32 Kriteria Penilaian *Faithfulness*

Nilai	Kategori	Deskripsi
≥ 90%	Sangat Baik	Hampir semua pernyataan jawaban sesuai dengan dokumen sumber
80% - 89%	Baik	Sebagian besar pernyataan jawaban didukung dokumen
70% - 79%	Cukup	Masih ada beberapa pernyataan yang tidak sesuai dokumen
< 70%	Kurang	Banyak pernyataan jawaban tidak dapat diverifikasi

2. *Hallucination Rate*

Hallucination Rate mengukur tingkat informasi yang dibuat model tanpa dasar dari dokumen sumber. Semakin rendah nilainya, maka semakin baik.

Rumus :

$$HR = 1 - F$$

Keterangan :

HR = Hallucination Rate

F = Faithfulness

Berdasarkan pada pertanyaan bagian tindak lanjut perbaikan terdapat pada Gambar 5.5 hasil pengujian diperoleh rata-rata Hallucination Rate sebesar:

F = 73 %

$$HR = 1 - F$$

$$HR = 100\% - 73\%$$

$$HR = 17 \%$$

Pada Tabel 5.33 menjelaskan Kriteria Penilaian *Hallucination Rate* :

Tabel 5.33 Kriteria Penilaian *Hallucination Rate*

Nilai	Kategori	Deskripsi
$\leq 10\%$	Sangat Baik	Hampir semua pernyataan jawaban sesuai dengan dokumen sumber
11% - 15%	Baik	Sebagian besar pernyataan jawaban didukung dokumen
16% - 20%	Cukup	Masih ada beberapa pernyataan yang tidak sesuai dokumen
$> 20\%$	Kurang	Banyak pernyataan jawaban tidak dapat diverifikasi

3. *Context Precision*

Context Precision mengukur ketepatan sistem dalam mengambil konteks yang benar-benar diperlukan untuk menjawab pertanyaan. Semakin tinggi nilainya, semakin sedikit konteks tidak relevan yang diambil.

Rumus :

$$CP = \frac{K_r}{K_t} \times 100 \%$$

Keterangan :

CP = Context Precision (%)

Kr = Jumlah konteks relevan yang diambil sistem

Kt = Total seluruh konteks yang diambil sistem

Berdasarkan pada pertanyaan buat laporan triwulan terbaru terdapat pada Gambar 5.5 hasil pengujian diperoleh rata-rata Context Precision sebesar:

Kr = 25 konteks dari basis pengetahuan

Kt = 22 konteks dinilai relevan

$$CP = \frac{K_r}{K_t} \times 100 \%$$

$$CP = \frac{25}{22} \times 100 \%$$

$$CP = 88 \%$$

Pada Tabel 5.34 menjelaskan Kriteria Penilaian *Context Precision*:

Tabel 5.34 Kriteria Penilaian *Context Precision*

Nilai	Kategori	Deskripsi
$\geq 90\%$	Sangat Baik	Sistem sangat tepat, hampir semua konteks yang diambil diperlukan untuk menjawab pertanyaan
80% - 89%	Baik	Sistem cukup tepat, sebagian besar konteks yang diambil relevan
70% - 79%	Cukup	Masih ada beberapa konteks tidak relevan yang ikut diambil
$< 70\%$	Kurang	Sistem banyak mengambil informasi yang tidak berguna/tidak relevan

4. *Context Recall* (Kelengkapan Konteks)

Context Recall (Kelengkapan Konteks) mengukur kemampuan sistem dalam menemukan seluruh informasi relevan yang tersedia di knowledge base. Semakin tinggi nilainya, semakin lengkap informasi yang berhasil diambil.

Rumus :

$$CR = \frac{P_d}{P_r} \times 100 \%$$

Keterangan :

CR = Context Recall (%)

Pd = Jumlah pernyataan referensi yang didukung oleh konteks yang diambil sistem

Pr = Total pernyataan dalam referensi

Berdasarkan pada pertanyaan buatlah laporan triwulan terbaru terdapat pada Gambar 5.5 hasil pengujian diperoleh rata-rata Context Recall sebesar:

Pr = 11 total informasi penting

Pd = 10 informasi yang berhasil ditemukan oleh sistem

$$CR = \frac{P_d}{P_r} \times 100 \%$$

$$CR = \frac{10}{11} \times 100 \%$$

$$CR = 90,91 \% \approx 91 \%$$

Pada Tabel 5.35 menjelaskan Kriteria Penilaian *Context Recall*:

Tabel 5.35 Kriteria Penilaian Context Recall (Kelengkapan Konteks)

Nilai	Kategori	Deskripsi
$\geq 90\%$	Sangat Baik	Sistem sangat lengkap, hampir semua informasi diambil
80% - 89%	Baik	Sebagian besar informasi berhasil ditemukan
70% - 79%	Cukup	Masih ada beberapa informasi penting yang terlewat
$< 70\%$	Kurang	Banyak informasi penting tidak ditemukan

5. *F1 Score* (Keseimbangan Presisi dan Kelengkapan)

F1 Score (Keseimbangan Presisi dan Kelengkapan) mengukur keseimbangan antara Context Precision (ketepatan) dan Context Recall (kelengkapan). Sistem yang baik harus memiliki keduanya yang tinggi.

Rumus:

$$F1 = \frac{2 \times (CP \times CR)}{CP + CR} \times 100 \%$$

F1 = F1 Score (%)

CP = Context Precision

CR = Context Recall

Berdasarkan pada pertanyaan buatlah laporan triwulan terbaru terdapat pada Gambar 5.5 hasil pengujian diperoleh rata-rata FI Score sebesar:

Context Precision (CP) = 88%

Context Recall (CR) = 91%

$$FI = \frac{2 \times (CP \times CR)}{CP + CR} \times 100 \%$$

$$FI = \frac{2 \times (88 \times 91)}{88 + 91} \times 100 \%$$

$$FI = 80 \%$$

Pada Tabel 5.36 menjelaskan Kriteria Penilaian F1 Score:

Tabel 5.36 Penilaian F1 Score

Nilai	Kategori	Deskripsi
$\geq 90\%$	Sangat Baik	Sistem sangat seimbang, tepat dan lengkap
80% - 89%	Baik	Sistem cukup seimbang, performa retrieval solid
70% - 79%	Cukup	Masih timpang antara ketepatan dan kelengkapan
$< 70\%$	Kurang	Keseimbangan buruk, perlu perbaikan signifikan

6. RAGAS Score (Skor Keseluruhan)

RAGAS Score (Skor Keseluruhan) Metrik komposit yang menggabungkan semua metrik RAGAS (Faithfulness, Context Precision, Context Recall, dan Answer Relevancy) untuk memberikan penilaian komprehensif terhadap kualitas sistem RAG secara keseluruhan.

Rumus:

$$RS = \frac{F + CP + CR + AR}{4}$$

Keterangan;

RS = RAGAS Score

F = Faithfulness

CP = Context Precision

CR = Context Recall

AR = Answer Relevancy

Berdasarkan rata - rata terdapat pada Gambar 5.5 hasil pengujian diperoleh rata-rata FI Score sebesar:

Faithfulness (F) = 76,0%

Context Precision (CP) = 79,0%

Context Recall (CR) = 82,0%

Answer Relevancy (AR) = 79,8%

$$RS = \frac{F + CP + CR + AR}{4}$$

$$RS = \frac{72 + 79 + 82 + 79,2}{4}$$

$$RS = 79,2 \%$$

Pada Tabel 5.37 menjelaskan Kriteria Penilaian RAGAS Score:

Tabel 5.37 Kriteria Penilaian RAGAS Score

Nilai	Kategori	Deskripsi
$\geq 90\%$	Sangat Baik	Sistem RAG berkualitas sangat tinggi, semua aspek bekerja optimal
80% - 89%	Baik	Sistem RAG handal, siap digunakan dengan sedikit catatan
70% - 79%	Cukup	Sistem RAG berfungsi baik namun masih perlu optimalisasi
$< 70\%$	Kurang	Sistem RAG perlu perbaikan signifikan di berbagai aspek

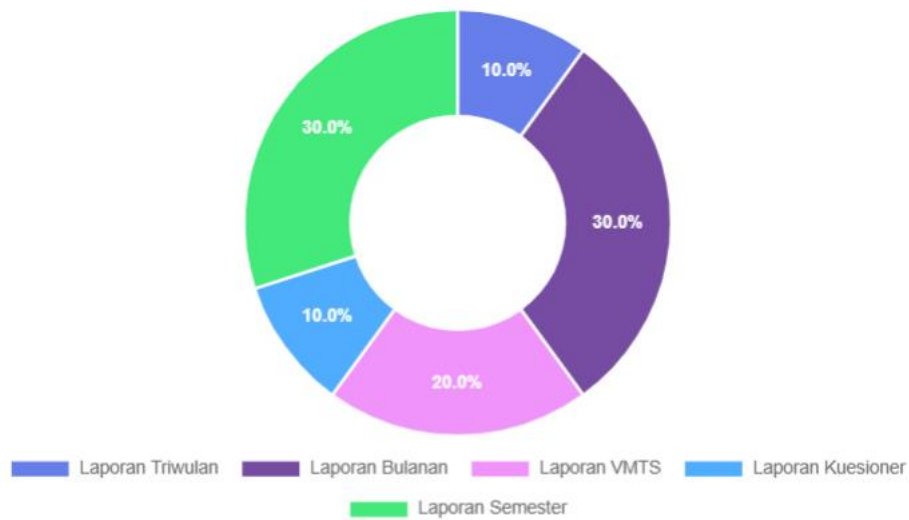
Hasil Evaluasi Per Skenario Pengujian

NO	PERTANYAAN / SKENARIO	KATEGORI	FAITH.	HALLU.	PREC.	RECALL	F1
1	Buat laporan triwulan terbaru	Laporan Triwulan	74.0%	15.0%	88.0%	91.0%	80.0%
2	perbaiki bagian pendahuluan	Laporan Bulanan	76.0%	10.0%	80.0%	87.0%	83.0%
3	Buatkan laporan VMTS semester ini	Laporan VMTS	73.0%	12.0%	78.0%	79.0%	73.0%
4	bagian tindak lanjut perbaiki	Laporan Bulanan	73.0%	17.0%	70.0%	78.0%	74.0%
5	Buatkan laporan kuesioner terbaru	Laporan Kuesioner	80.0%	11.0%	79.0%	84.0%	81.0%
6	perbaiki bagian pelaksanaan dan ubah jadi 2026	Laporan Semester	79.0%	14.0%	75.0%	76.0%	70.0%
7	Buatkan laporan artefak bulanan	Laporan Bulanan	76.0%	13.0%	82.0%	89.0%	85.0%
8	Ringkas visi, misi, dan tujuan Program Studi TRPL	Laporan VMTS	73.0%	13.0%	87.0%	72.0%	89.0%
9	Buatkan laporan semester genap terbaru	Laporan Semester	81.0%	13.0%	75.0%	83.0%	79.0%
10	pada bagian rekomendasi masih belum bagus coba perbaiki lagi	Laporan Semester	70.0%	16.0%	77.0%	77.0%	72.0%
Rata-rata			76.0%	13.0%	79.0%	82.0%	79.0%
RAGAS Score			79.2%				

Gambar 5.5 Hasil Evaluasi RAGAS Per Skenario Pengujian

Pada Gambar 5.5 menampilkan Hasil Evaluasi RAGAS Per Skenario Pengujian. Sistem RAG menunjukkan performa cukup baik dengan skor keseluruhan 79,2%. Aspek Kelengkapan informasi (Recall 82%) namun masih perlu ditingkatkan pada aspek kesesuaian jawaban (Faithfulness 76%) dan ketepatan konteks (Precision 79%)

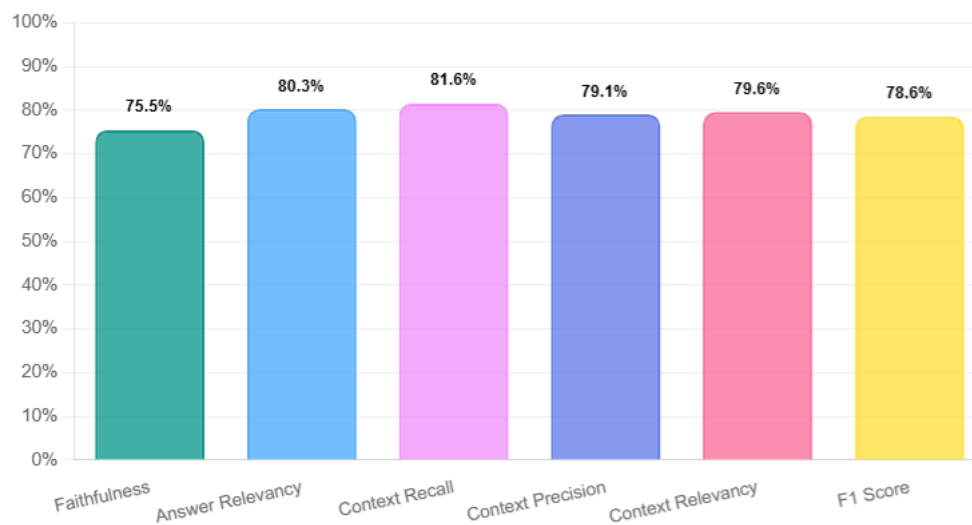
Distribusi Kategori Pengujian



Gambar 5.6 Distribusi Kategori Pengujian

Pada Gambar 5.6 menampilkan Distribusi Kategori Pengujian di atas menunjukkan komposisi skenario pengujian berdasarkan kategori. Dari total 10 skenario, proporsi terbanyak adalah Laporan Triwulan (30%) , diikuti Laporan Bulanan (20%) . Sisanya tersebar merata pada kategori Laporan VMTS, Kuesioner, dan Semester masing-masing (10%)

Perbandingan Metrik RAGAS



Gambar 5.7 Perbandingan Metrik RAGAS

Pada Gambar 5.7 menampilkan perbandingan metrik RAGAS. Hasil pengujian menunjukkan bahwa Context Recall memperoleh nilai tertinggi sebesar 81,6%, sedangkan Faithfulness memperoleh nilai terendah sebesar 75,5%. Secara keseluruhan, sistem menunjukkan performa yang cukup baik dengan seluruh metrik berada di atas 75%.

2.5. Kesimpulan dan Insight

Berdasarkan hasil pengujian dan implementasi, sistem yang dikembangkan menunjukkan bahwa teknologi NLP mampu memahami instruksi pengguna dengan baik, sementara pendekatan Retrieval-Augmented Generation (RAG) berhasil meningkatkan akurasi informasi yang diberikan. Selain itu, penggunaan AI Agent dapat mengotomatisasi proses penyusunan dokumen sehingga meningkatkan efisiensi kerja. Temuan ini menunjukkan bahwa pemanfaatan AI berpotensi mendukung proses penjaminan mutu akademik, dengan kualitas jawaban yang sangat dipengaruhi oleh efektivitas mekanisme retrieval dalam memperoleh informasi yang relevan dan akurat.

BAB III IMPLEMENTASI BIG DATA

3.1. Latar Belakang

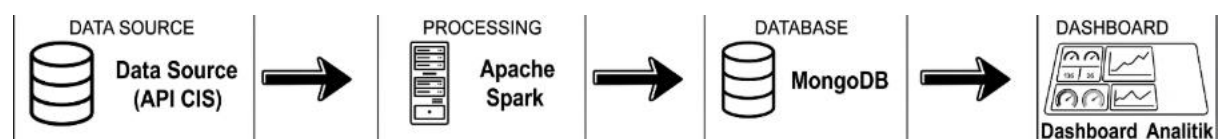
Data kuesioner evaluasi pembelajaran mahasiswa di Fakultas Vokasi Institut Teknologi Del terus bertambah setiap semester. Sistem analisis yang ada tidak lagi mampu menangani volume data tersebut secara efisien, sehingga dibutuhkan sistem analitik berbasis Big Data yang mampu memproses data dalam skala besar secara terdistribusi.

Permasalahan utama yang dihadapi meliputi:

- Volume data kuesioner yang terus bertambah setiap semester melebihi kapasitas analisis SQL konvensional.
- Proses analisis manual yang lambat dan tidak konsisten antar periode.
- Sulitnya monitoring kualitas pembelajaran secara real-time lintas program studi.
- Data yang bersifat semi-terstruktur (JSON dari kuesioner) tidak optimal jika disimpan di database relasional murni.

Untuk mengatasi permasalahan tersebut, dibangun sistem Big Data terintegrasi menggunakan Apache Spark sebagai engine pemrosesan terdistribusi, MySQL sebagai database relasional untuk data master, MongoDB sebagai database NoSQL untuk hasil analisis agregat, dan Laravel Dashboard sebagai antarmuka visualisasi interaktif.

Pengolahan data kuesioner pada sistem menggunakan Apache Spark sebagai *engine* pemrosesan data untuk mendukung analisis data kuesioner mahasiswa secara efisien dan terukur. Data kuesioner yang diperoleh dari API *Campus Information System* (CIS) diproses melalui pipeline yang terdiri dari beberapa komponen yang bekerja secara terintegrasi.



Gambar Pipeline Pengolahan Data Kuesioner

Pada Gambar menampilkan Pipeline Pengolahan Data Kuesioner, Pada pipeline tersebut, Apache Spark bertugas melakukan proses ingestion, cleaning, transformasi, dan agregasi data kuesioner sebelum hasil analisis disimpan ke MongoDB. Data yang telah diolah kemudian digunakan oleh dashboard analitik untuk menampilkan informasi, visualisasi, dan insight yang mendukung proses monitoring serta evaluasi kualitas pembelajaran. Setiap komponen dalam pipeline memiliki peran yang spesifik dalam memastikan data kuesioner dapat diproses, dianalisis, dan divisualisasikan secara akurat, konsisten, dan efisien.

2.6. Data Set

Data Source merupakan tahap awal dalam pipeline pengolahan data kuesioner, yaitu proses pengambilan data mentah yang akan dianalisis oleh sistem. Data diperoleh dari *Campus Information System (CIS)* Institut Teknologi Del melalui API yang disediakan oleh sistem akademik kampus. Data yang diperoleh kemudian digunakan sebagai sumber utama dalam proses analisis kualitas pembelajaran.

Data yang masuk ke dalam pipeline terdiri dari data terstruktur dan semi-terstruktur yang mencakup informasi kuesioner mahasiswa, dosen, mata kuliah, program studi, serta metadata pendukung lainnya. Volume data yang diproses bergantung pada jumlah mahasiswa yang mengisi kuesioner pada setiap semester serta data historis yang tersedia pada CIS.

No	API Endpoint	HTTP Method	Description
1	/auth/login	POST	Authenticate users and generate a JWT token.
2	/lecturers	GET	Retrieve the list of lecturers.
3	/academic-periods/active	GET	Retrieve the currently active academic period.
4	/courses	GET	Retrieve the list of courses based on study program, semester, and academic year.
5	/schedules/lecture	GET	Retrieve the teaching schedule for a specific lecturer.
6	/monitoring/practicum	GET	Monitor practicum materials based on class and academic period.
7	/monitoring/theory	GET	Monitor theory materials based on class and academic period.
8	/questionnaires/summary	GET	Retrieve the questionnaire result summary based on the questionnaire ID.

Sumber data yang digunakan dalam sistem meliputi:

1. Data Kuesioner Mahasiswa: Berisi jawaban dan penilaian mahasiswa terhadap proses pembelajaran, dosen, materi perkuliahan, serta aspek-aspek evaluasi lainnya yang dikumpulkan pada setiap akhir semester.

2. Data Dosen: Berisi informasi dosen yang digunakan sebagai dimensi analisis untuk menghitung tingkat kepuasan mahasiswa, performa dosen, dan rekapitulasi hasil evaluasi pembelajaran.
3. Data Matakuliah: Berisi informasi mata kuliah yang digunakan untuk mengelompokkan dan menganalisis hasil kuesioner berdasarkan mata kuliah yang diampu.
4. Data Program Studi: Berisi informasi program studi yang digunakan untuk menghasilkan analisis dan perbandingan tingkat kepuasan mahasiswa antar program studi di Fakultas Vokasi.
5. Data Periode Akademik: Berisi informasi semester dan tahun akademik yang digunakan untuk analisis tren kepuasan mahasiswa dari waktu ke waktu.

Data-data tersebut diambil secara berkala melalui API CIS dan selanjutnya diproses menggunakan Apache Spark untuk melakukan pembersihan data, transformasi, agregasi, serta perhitungan statistik sebelum disimpan ke MongoDB sebagai basis data analitik

2.7. Apache Spark

Apache Spark digunakan sebagai engine pemrosesan data yang bertanggung jawab untuk melakukan pengolahan, transformasi, dan analisis data kuesioner mahasiswa. Pemilihan Apache Spark didasarkan pada kemampuannya dalam memproses data secara paralel dan terdistribusi sehingga mampu menangani proses analisis data dalam jumlah besar dengan lebih efisien dibandingkan pendekatan query konvensional. Dalam pipeline pengolahan data kuesioner, Apache Spark memiliki beberapa fungsi utama sebagai berikut:

a. Data Ingestion

Data evaluasi mahasiswa diambil dari database MySQL menggunakan Apache Spark JDBC connector. Proses ingestion dilakukan secara paralel dan read-only agar tidak mengganggu stabilitas database utama. Kode ingestion menggunakan `spark.read.format("jdbc")` dengan parameter koneksi ke tabel jawaban_kuesioner. Data yang berhasil di-ingest disimpan ke collection KUESIONER_MOGOS di MongoDB berisi: raw JSON kuesioner, metadata dokumen, dan statistik kuesioner per mata kuliah.

```
# =====  
# READ DATA MONGO  
# =====  
pipeline = ""  
[  
  {  
    "$match": {  
      "is_analyzed": false  
    }  
  }  
]
```

```

]
"""

df = spark.read \
    .format("mongodb") \
    .option("database", "gkm_chatbot") \
    .option("collection", "kuesioner_mongos") \
    .option("aggregation.pipeline", pipeline) \
    .load()

```

Implementasi Data Ingestion dalam mengambil data dari API Campus Information System (CIS) dan memuatnya ke dalam Spark DataFrame sebagai struktur data utama yang digunakan dalam proses analisis.

b. Data Cleaning

```

# =====
# AMBIL DOSEN DARI JUDUL
# contoh:
# (AMS/D4TRPL)
# =====
df = df.withColumn(
    "dosen_pengajar",
    regexp_extract(
        col("judul_kuesioner"),
        r"\((([A-Z]{3})\V[^()]*\V)$",
        1
    )
)

```

```

# =====
# EXPLODE PRODI
# =====
df = df.withColumn(
    "prodi_item",
    explode_outer(col("prodi"))
)

df = df.withColumn(
    "prodi",
    col("prodi_item")
)

df = df.withColumn(
    "jenis_kuesioner",
    col("jenis_kuesioner")
)

```

Implementasi Data Cleaning dalam membersihkan data dari nilai kosong (null), data duplikat, inkonsistensi format, serta melakukan normalisasi terhadap atribut yang diperlukan agar data siap diproses lebih lanjut.

c. Data Transformation

```
# =====
# EXPLODE REKAPITULASI
# =====
rekap = df.select(
    col("kuesioner_id"),
    col("kode_mk"),
    col("judul_kuesioner"),
    col("dosen_pengajar"),
    col("jenis_kuesioner"),
    # col("kode_prodi"),
    col("prodi"),
    col("periode").alias("tahun"),
    col("semester"),
    explode(col("raw_data.statistik.rekapitulasi")).alias("rekap")
)

# =====
# EXPLODE RINCIAN JAWABAN
# =====
detail = rekap.select(
    col("kuesioner_id"),
    col("kode_mk"),
    col("judul_kuesioner"),
    col("dosen_pengajar"),
    col("jenis_kuesioner"),
    # col("kode_prodi"),
    col("prodi"),
    col("tahun"),
    col("semester"),

    regexp_replace(
        col("rekap.pertanyaan"),
        "<[^>]+>",
        ""
    ).alias("pertanyaan"),

    col("rekap.total_suara")
        .cast("int")
        .alias("total_suara"),

    explode(
        col("rekap.rincian_jawaban")
    )
)
```

```
).alias("jawaban")
```

Implementasi Data Cleaning dalam membersihkan data dari nilai kosong (null), data duplikat, inkonsistensi format, serta melakukan normalisasi terhadap atribut yang diperlukan agar data siap diproses lebih lanjut.

```
# =====  
# AMBIL NILAI & JUMLAH  
# =====  
hasil = detail.select(  
    col("kuesioner_id"),  
    col("kode_mk"),  
    col("judul_kuesioner"),  
    col("dosen_pengajar"),  
    col("jenis_kuesioner"),  
    # col("kode_prodi"),  
    col("prodi"),  
    col("tahun"),  
    col("semester"),  
    col("pertanyaan"),  
    col("total_suara"),  
  
    col("jawaban.jawaban")  
        .cast("int")  
        .alias("jawaban"),  
  
    col("jawaban.jumlah")  
        .cast("int")  
        .alias("jumlah")  
)
```

Implementasi Data Aggregation untuk melakukan pengelompokan data menggunakan fungsi `groupBy()` berdasarkan berbagai dimensi analisis seperti dosen, mata kuliah, program studi, dan semester untuk menghasilkan data rekapitulasi yang siap digunakan pada dashboard.

e. Statistical Analysis

```
# =====  
# KATEGORI HASIL  
# =====  
analisis = analisis.withColumn(  
    "kategori_hasil",
```

```
when(col("rata_rata") >= 3.1, "Sangat Baik")
.when(col("rata_rata") >= 2.1, "Baik")
.when(col("rata_rata") >= 1.1, "Cukup")
.otherwise("Kurang")
)
```

Implementasi Statistical Analysis untuk menghasilkan berbagai metrik statistik yang digunakan dalam proses evaluasi pembelajaran, seperti rata-rata nilai kepuasan, jumlah responden, dan persentase tingkat kepuasan.

```
# =====
# LABEL SENTIMENT
# =====
analysis = analysis.withColumn(
    "sentiment",

    when(
        col("persentase_kepuasan") >= 85,
        "positive"
    )
    .when(
        col("persentase_kepuasan") >= 60,
        "neutral"
    )
    .otherwise("negative")
)
```

implementasi *Statistical Analysis* untuk menghasilkan berbagai metrik statistik yang digunakan dalam proses evaluasi pembelajaran, seperti rata-rata nilai kepuasan, jumlah responden, dan persentase tingkat kepuasan.

Melakukan transformasi data seperti ekstraksi informasi, konversi tipe data, normalisasi nilai kuesioner, serta penggabungan data dari berbagai sumber yang relevan. Selain melakukan transformasi dan agregasi data, Apache Spark juga melakukan perhitungan statistik yang menjadi dasar penyusunan dashboard analitik.

a. Transformasi Skor Kuesioner

Sebelum dilakukan perhitungan statistik, nilai jawaban kuesioner terlebih dahulu ditransformasikan agar skor yang lebih tinggi merepresentasikan tingkat kepuasan yang lebih tinggi. Transformasi dilakukan sebagai berikut:

Nilai Asli	Kategori (SS)	Skor Hasil Transformasi
1	Sangat Setuju (SS)	4
2	Setuju (S)	3
3	Cukup Setuju (CS)	2

4	Tidak Setuju (TS)	1

b. Perhitungan Weighted Score

Setelah proses transformasi skor selesai dilakukan, sistem menghitung Weighted Score untuk setiap kategori jawaban. Perhitungan ini bertujuan untuk mempertimbangkan jumlah responden yang memilih setiap pilihan jawaban sehingga hasil analisis dapat merepresentasikan kondisi sebenarnya dengan lebih akurat.

Keterangan:

WS_i = Weighted Score pada kategori jawaban ke-i

S_i = Skor hasil transformasi pada kategori jawaban ke-i

J_i = Jumlah responden yang memilih kategori jawaban ke-i

```
# =====
# HITUNG WEIGHTED SCORE
# =====
hasil = hasil.withColumn(
  "weighted_score",
  col("skor") * col("jumlah")
)
```

re diperoleh dengan mengalikan skor hasil transformasi dengan jumlah responden yang memilih kategori jawaban tertentu. Semakin tinggi skor dan semakin banyak responden yang memilih kategori tersebut, maka semakin besar kontribusinya terhadap nilai kepuasan secara keseluruhan.

c. Perhitungan Rata-Rata Kepuasan

```
# =====
# ANALISIS PER PERTANYAAN
# =====
analisis = hasil.groupBy(
  "kuesioner_id",
  "kode_mk",
  "judul_kuesioner",
  "dosen_pengajar",
  "jenis_kuesioner",
  "prodi",
  # col("kode_prodi"),
  "tahun",
  "semester",
  "pertanyaan",
  "total_suara"
).agg(

  sum("jumlah")
  .alias("total_jawaban"),
```

```
sum("weighted_score")
  .alias("total_skor")
)
```

Pada Gambar diatas menampilkan implementasi Perhitungan Rata-Rata Tingkat kepuasan dihitung menggunakan nilai rata-rata (mean) dari seluruh skor responden. Rata-rata memberikan gambaran umum tingkat kepuasan mahasiswa terhadap aspek yang dievaluasi.

Perhitungan ini dilakukan dengan rumus :

$$\bar{X} = \frac{\sum_{i=1}^n WS_i}{\sum_{i=1}^n Ji}$$

Keterangan:

$\bar{x}$ = rata-rata tingkat kepuasan

WSi = Weighted Score pada kategori jawaban ke-i

Ji = jumlah responden pada kategori jawaban ke-i

n = jumlah kategori jawaban

d. Perhitungan Presentasi Kepuasan

```
# =====
# HITUNG PERSENTASE
# =====
analisis = analisis.withColumn(
  "persentase_kepuasan",

  round(
    (col("rata_rata") / 4) * 100,
    2
  )
)
```

Pada Gambar 4.39 menampilkan implementasi untuk Perhitungan Presentasi Kepuasan untuk mempermudah interpretasi hasil analisis, rata-rata skor kemudian dikonversi ke dalam bentuk persentase kepuasan.

$$P = \frac{\bar{x}}{4} \times 100 \%$$

Keterangan :

P = Persentase Kepuasan

$\bar{x}$ = rata-rata tingkat kepuasan

4 = skor maksimum hasil transformasi

Hasil persentase ini digunakan pada dashboard analitik untuk mempermudah interpretasi tingkat kepuasan mahasiswa terhadap dosen, mata kuliah, program studi, maupun periode akademik tertentu.

Hasil persentase ini digunakan pada dashboard analitik untuk mempermudah interpretasi tingkat kepuasan mahasiswa terhadap dosen, mata kuliah, program studi, maupun periode akademik tertentu.

```
# =====  
# HITUNG PERSENTASE  
# =====  
analisis = analisis.withColumn(  
    "persentase_kepuasan",  
  
    round(  
        (col("rata_rata") / 4) * 100,  
        2  
    )  
)
```

2.8. MongoDB

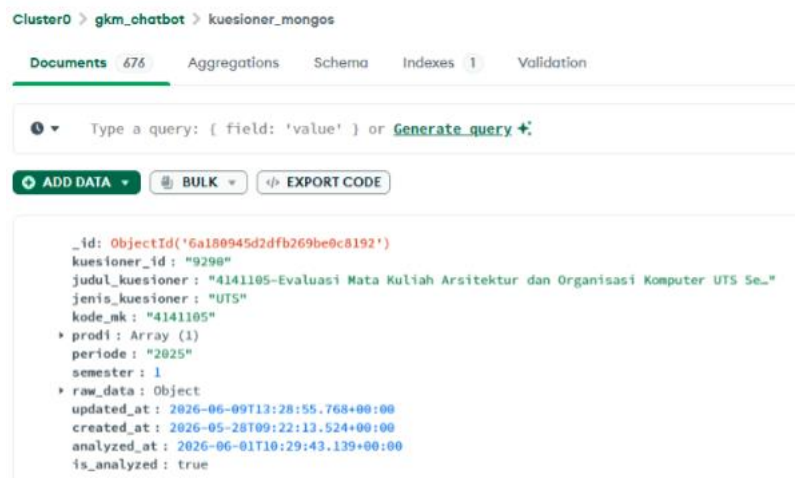
MongoDB digunakan sebagai database analitik yang menyimpan hasil pengolahan data dari Apache Spark. Pemilihan MongoDB didasarkan pada kemampuannya dalam menyimpan data semi-terstruktur dalam bentuk dokumen JSON yang fleksibel, sehingga sesuai untuk menyimpan hasil analisis yang memiliki struktur dinamis. Dalam arsitektur sistem, MongoDB berperan sebagai penyimpanan utama hasil transformasi, agregasi, dan perhitungan statistik sebelum ditampilkan pada dashboard analitik.

1. Struktur Database

Database utama yang digunakan adalah gkm_chatbot yang menyimpan beberapa collection utama, yaitu data kuesioner mentah, hasil analisis detail, summary mata kuliah, serta cache response AI. Data ini digunakan untuk mendukung proses analitik dan visualisasi pada dashboard sistem.

2. Collection Utama

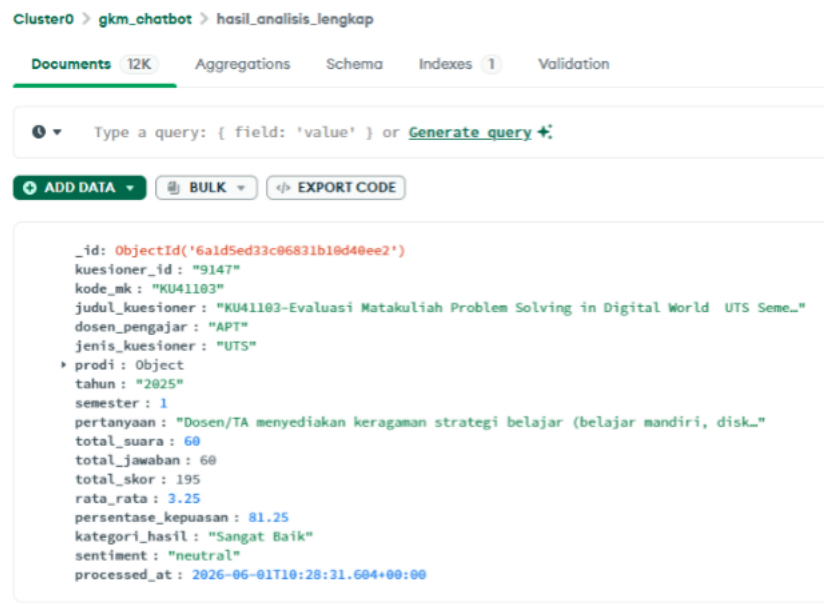
a. Kuesioner_mongos



Gambar 4.40 Kuesioner Mongos

Pada Gambar 4.40 menampilkan Kuesioner Mongos. Collection ini menyimpan data mentah kuesioner yang diperoleh dari API Campus Information System (CIS). Data ini menjadi input utama bagi Apache Spark dalam proses pengolahan. Setiap dokumen berisi metadata kuesioner, informasi mata kuliah, program studi, serta data statistik jawaban mahasiswa dalam bentuk nested JSON.

b. Hasil_analisis lengkap

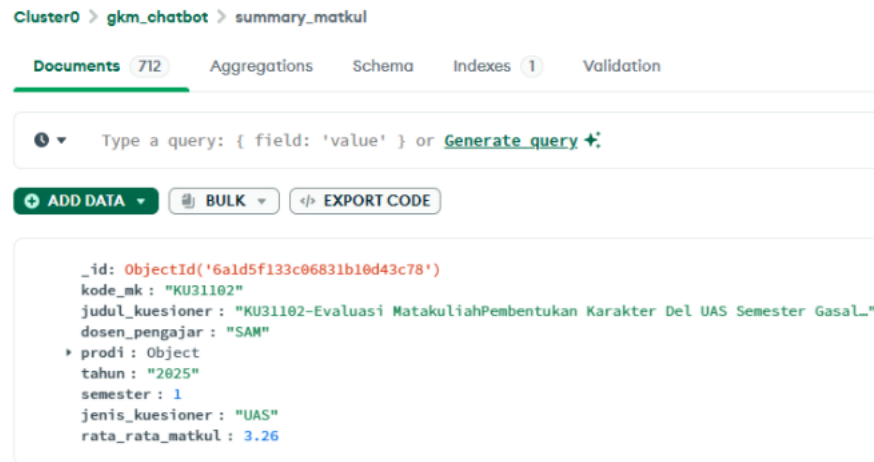


Gambar 4.41 Hasil Analisis Lengkap

Pada Gambar 4.41 menampilkan Hasil Analisis Lengkap. Collection ini menyimpan hasil analisis detail yang dihasilkan oleh Apache Spark. Data mencakup perhitungan total skor, rata-rata, persentase kepuasan, kategori hasil, serta sentiment analysis pada setiap pertanyaan kuesioner.

c. Summary_matkul

Collection ini menyimpan hasil agregasi pada level mata kuliah, yang digunakan untuk kebutuhan dashboard overview dan perbandingan antar mata kuliah berdasarkan nilai rata-rata dan tingkat kepuasan.



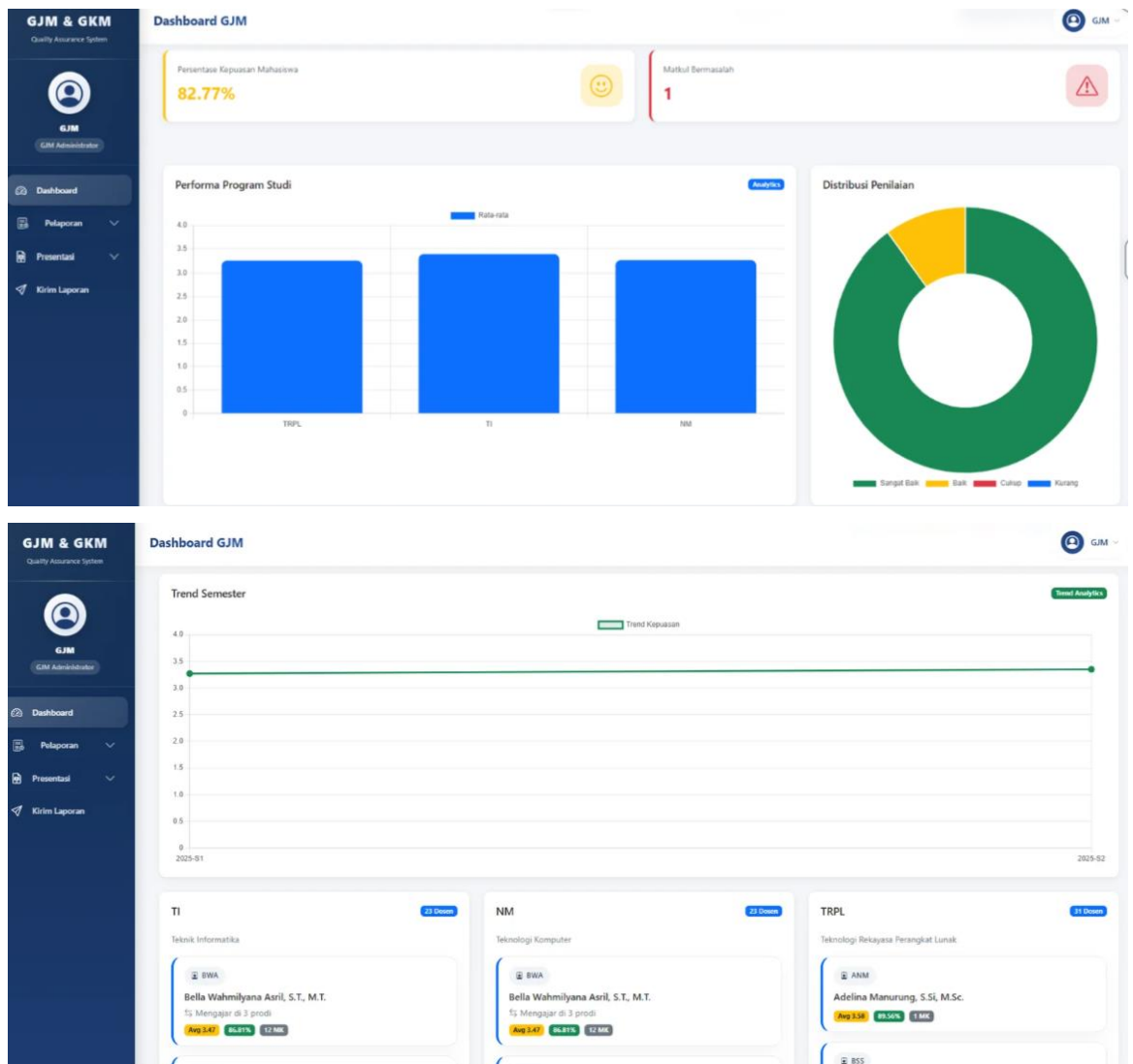
Gambar 4.42 Summary Matakuliah

Pada Gambar 4.42 menampilkan Summary Mata Kuliah. MongoDB berperan sebagai database analitik utama dalam sistem karena memiliki fleksibilitas tinggi dalam menyimpan data semi-terstruktur, mendukung nested document, serta memiliki performa yang baik dalam operasi agregasi. Integrasi dengan Apache Spark memungkinkan proses analitik berjalan secara efisien, terstruktur, dan scalable untuk mendukung kebutuhan dashboard analitik sistem kuesioner.

3.2. Hasil dan Analisis

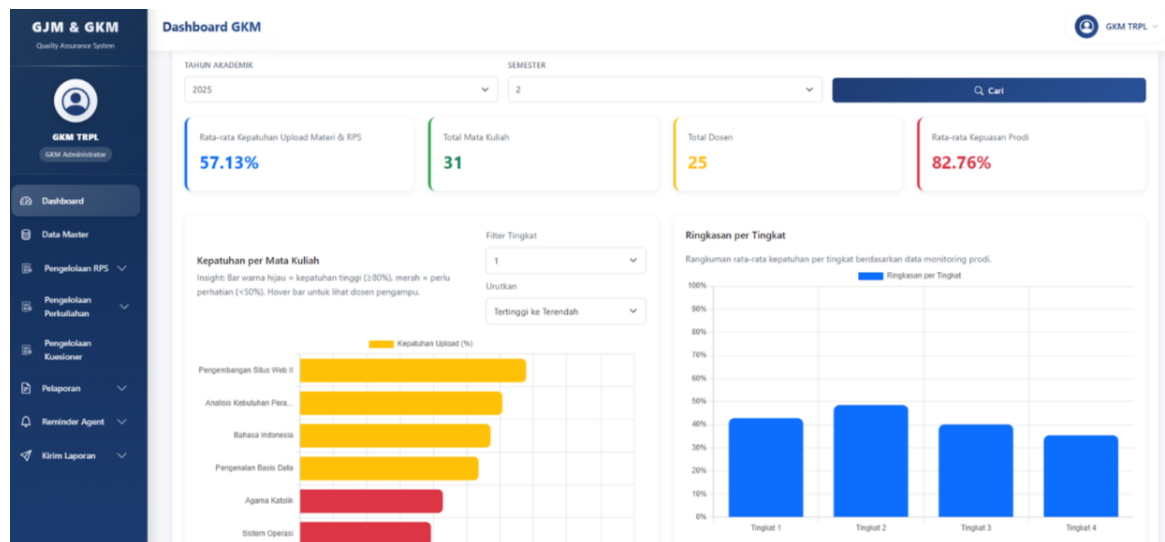
Sistem Big Data yang dibangun berhasil memproses dan menganalisis data kuesioner mahasiswa secara otomatis. Hasil analisis ditampilkan melalui dashboard analitik interaktif berbasis Laravel dan Chart.js. Berikut adalah tampilan-tampilan dashboard yang dihasilkan:

- a. Visualisasi Dashboard GJM



Insight dan Analisis Hasil

- Pada grafik Performa Program Studi, program studi TI memiliki nilai rata-rata tertinggi, diikuti NM dan TRPL. Namun, perbedaan nilai antar program studi relatif kecil sehingga kualitas layanan akademik cukup merata.
 - Grafik Trend Semester menunjukkan adanya peningkatan tipis tingkat kepuasan dari Semester ganjil ke Semester genap tahun 2025. Meskipun tidak signifikan, tren ini mengindikasikan perbaikan kualitas layanan akademik yang konsisten.
 - Grafik Distribusi Penilaian menunjukkan mayoritas respons mahasiswa berada pada kategori "Sangat Baik", dengan sebagian kecil berada pada kategori "Baik", serta hampir tidak ada penilaian negatif.
- b. Visualisasi Dashboard GKM



Insight dan Analisis Hasil

- Rata-rata kepatuhan upload materi dan RPS masih rendah, yaitu 57,13%, menunjukkan bahwa masih banyak mata kuliah yang belum memenuhi target kelengkapan dokumen perkuliahan.
- Rata-rata kepuasan program studi mencapai 82,76%, mengindikasikan bahwa tingkat kepuasan mahasiswa terhadap proses pembelajaran dan layanan akademik tergolong baik.
- Beberapa mata kuliah masih berada pada kategori perlu perhatian (<50%), seperti Agama Katolik dan Sistem Operasi, sehingga memerlukan tindak lanjut dari dosen pengampu.

3.3. Kesimpulan

Sistem Big Data terintegrasi berhasil dibangun menggunakan Apache Spark sebagai engine pemrosesan terdistribusi. Data kuesioner mahasiswa berhasil diproses secara otomatis melalui pipeline ingestion, cleaning, transformasi, dan penyimpanan. Integrasi SQL (MySQL) dan NoSQL (MongoDB) berjalan sesuai rancangan, memanfaatkan kekuatan masing-masing paradigma penyimpanan. Dashboard analitik berbasis Laravel berhasil menampilkan analisis kualitas pembelajaran secara interaktif dan real-time.

Meskipun volume data yang dapat diakuisisi pada periode pengembangan ini terbatas pada data 2 tahun terakhir (± 57.600 baris) karena cakupan API CIS yang tersedia, sistem telah terbukti berjalan dengan baik dan siap untuk menangani volume data yang lebih besar seiring dengan pertumbuhan data historis maupun perluasan akses API di masa mendatang.

BAB IV PENUTUP

4.1. Kesimpulan dan Saran

4.1.1. Kesimpulan

Sistem web agent berbasis Laravel 12 berhasil dibangun untuk mengotomatisasi administrasi GKM dan GJM Fakultas Vokasi IT Del, mencakup dua pilar utama: pipeline Big Data terdistribusi dan pipeline NLP-RAG.

Pipeline NLP-RAG berhasil diimplementasikan dengan skor RAGAS keseluruhan 77,94%, Faithfulness 87,5%, dan Ambiguity Rate hanya 15,38%.

Optimasi konfigurasi RAG berhasil menurunkan waktu respons 65% (dari 37,41 detik menjadi 12,97 detik) sambil mempertahankan kualitas output.

Pipeline Big Data menggunakan Apache Spark berhasil memproses data kuesioner secara terdistribusi dengan integrasi SQL-NoSQL yang efisien.

Waktu penyusunan laporan akademik berhasil dipangkas dari rata-rata 3,5 jam menjadi di bawah 5 menit, meningkatkan efisiensi kerja tim GKM/GJM secara signifikan.

4.1.2. Saran

Pengembangan model embedding khusus bahasa Indonesia akademik untuk meningkatkan kualitas semantic retrieval.

Implementasi multi-agent system untuk penanganan paralel berbagai jenis laporan secara simultan.

Eksplorasi teknik advanced RAG (HyDE, cross-encoder re-ranking) untuk meningkatkan Context Precision.

Perluasan akses API CIS untuk data historis lebih dari 2 tahun, sehingga sistem dapat diuji pada skala data penuh (>100.000 records).

4.2. Pembagian Taks Perorang

LAMPIRAN

Lampiran Repository

- Link GitHub Repository: <https://github.com/RaySimarmata/PA3-KEL06-2026>